

Week 2 Hands-On Lab – Full Solutions & Explanations

PART 1: Fixed Secure API

```
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const rateLimit = require('express-rate-limit');
const app = express();
app.use(express.json());

// ✅ FIX 1: secret comes from environment, never hardcoded
const SECRET = process.env.JWT_SECRET; // set via `export JWT_SECRET=...`

const employees = {
  "101": { name: "Asha", department: "Engineering", salary: 90000, manager: "102" },
  "102": { name: "Ravi", department: "Sales", salary: 75000, manager: "201" },
};

// ✅ FIX 2: passwords stored as bcrypt hashes, never plaintext
// (hashes generated once at setup time, e.g. via bcrypt.hashSync("StrongPassword", 10))
const users = {
  "asha@company.com": { passwordHash: "$2b$10$exampleHashHere...", id: "101" },
  "admin@company.com": { passwordHash: "$2b$10$exampleHashHere...", id: "102" },
  "raj.manager@company.com": { passwordHash: "$2b$10$exampleHashHere...", id: "201" },
};

// ✅ Rate limiting – prevents brute force / MFA-fatigue-style abuse on login
const loginLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 5 });

// ✅ Simple audit log middleware – logs every request with actor + outcome
function auditLog(req, res, next) {
  const original = res.json;
  res.json = function (body) {
    console.log(JSON.stringify({
      actor: req.user ? req.user.id : "anonymous",
      outcome: body ? "success" : "failure",
    }));
    original.call(this, body);
  };
  next();
}
```

```

    role: req.user ? req.user.role : null,
    action: `${req.method} ${req.path}`,
    target_id: req.params.id || null,
    outcome: res.statusCode < 400 ? "success" : "denied",
    timestamp: new Date().toISOString()
  }));
  return original.call(this, body);
};
next();
}
app.use(auditLog);

// Auth middleware – verifies token, attaches user to request
function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization || "";
  const token = authHeader.replace("Bearer ", "");
  try {
    req.user = jwt.verify(token, SECRET); // { id, role }
    next();
  } catch (e) {
    res.status(401).json({ error: "invalid or expired token" });
  }
}

app.post('/login', loginLimiter, async (req, res) => {
  const { email, password } = req.body;
  const user = users[email];
  // Always run bcrypt.compare even on missing user, to avoid timing attacks
  const hash = user ? user.passwordHash : "$2b$10$invalidPlaceholderHash";
  const valid = await bcrypt.compare(password, hash);
  if (!user || !valid) {
    return res.status(401).json({ error: "invalid credentials" });
  }
  const token = jwt.sign({ id: user.id, role: user.role }, SECRET, { expiresIn: 1h });
  res.json({ token });
});

//  FIX 3: object-level AND property-level authorization enforced
app.get('/employee/:id', requireAuth, (req, res) => {
  const employee = employees[req.params.id];
  if (!employee) return res.status(404).json({ error: "not found" });

  const requester = req.user;
  const isSelf = requester.id === req.params.id;

```

```

const isManagerOfThisPerson = requester.role === "manager" && employee.
const isAdmin = requester.role === "admin";

if (!isSelf && !isManagerOfThisPerson && !isAdmin) {
  return res.status(403).json({ error: "not authorized to view this rec
}

// Property-level filtering: only self or admin sees salary
const canSeeSalary = isSelf || isAdmin;
const { salary, ...safeFields } = employee;
res.json(canSeeSalary ? employee : safeFields);
});

app.listen(3000, () => console.log('Secure HR bot API running on :3000'))

```

What changed and why (mapped back to Week 2 concepts)

Fix	Concept	Why it matters
<code>SECRET</code> from env var	Encryption/secrets management	Prevents credential leak via source control (recall the xAI GitHub key leak case)
bcrypt password hashing	Authentication	Even if the database is breached, passwords aren't directly usable (unlike McDonald's "123456")
Rate limiting on login	Secure APIs	Stops brute force and mirrors the MFA-fatigue defense we discussed
Short-lived JWT (1h vs 24h)	Authentication	Limits blast radius if a token is stolen – same logic as OAuth vs API keys
Ownership/role check before returning data	Authorization (BOLA)	Directly fixes the McDonald's Olivia / Meta chatbot vulnerability class
Salary field stripped unless self/admin	Property-level authorization	Fixes the "can see record but shouldn't see all fields" gap
Audit log middleware	Audit Logs	Every request now has actor, action, outcome, timestamp – enables forensics

One thing still missing (intentionally, for you to think about): this fix doesn't yet add the `on_behalf_of` + `triggering_prompt` fields we discussed for AI agents specifically. That's because this version assumes a *human* is calling directly. If an AI agent sat in front of this API, you'd add:

```

actor: "AI-HR-Bot",
on_behalf_of: req.user.id,
triggering_prompt: req.body.originalUserMessage

```

PART 2: McDonald's "Olivia" Breach – Answers

1. Which vulnerability from Part 1 maps directly to this real breach?

Fix 3 (BOLA) is the primary match – attackers could pull *any* applicant's record, including private chat transcripts, by manipulating an ID, exactly like the unfixed `/employee/:id` endpoint. Fix 2 (weak credentials) also matches directly – McDonald's exposed admin account used the password "123456," identical to the weak password in the original vulnerable code.

2. Do you agree this "wasn't advanced AI going off the rails"?

Yes – and this is the central lesson of the whole Secure APIs topic. The AI chatbot ("Olivia") behaved exactly as designed; it answered questions using data returned by its backend API. The failure was entirely in traditional API security – missing authorization checks and a forgotten test account – neither of which has anything to do with prompt injection, hallucination, or any AI-specific weakness. This reinforces the closing theme from our Secure APIs session: **AI failures are usually old vulnerabilities wearing a new interface.**

3. Which control would you prioritize checking before launch?

The strongest answer is **Authorization testing (specifically BOLA)** – because it's consistently the #1 most common API vulnerability (over 40% of all API vulnerabilities per 2026 data) and caused the largest real-world AI-related breach we discussed. A close second-priority answer: **Administration** – auditing for any leftover test/admin accounts before go-live, since that's a simple, checklist-level control that would have independently stopped this breach even if BOLA had somehow been caught.

4. Which of the 4 A's does the forgotten test account failure belong to?

Administration – this is a lifecycle failure, not an authentication or authorization *design* flaw. The account should have been deprovisioned (offboarded) once testing was complete. This is the same pattern as the Vercel/Context.ai case: a control that was fine *at setup* became a liability because nobody revisited it afterward.

PART 3: Scenario Decisions – Full Explanations

1. AI agent uses its own long-lived service key on behalf of any user → ❌ Not Secure

This is the "union vs. intersection" problem from our RBAC discussion. The agent's own broad access becomes a ceiling-less bypass – any user, even a low-privilege one, effectively borrows the service account's full power. Fix: forward the *user's own* short-lived, scoped token instead.

2. RAG chatbot retrieves everything, then relies on a system prompt to filter → ❌ Not Secure

This is the exact "retrieval-then-filter" flaw we covered under Authorization and again under Encryption. The sensitive data has already entered the model's context before any filtering happens – a cleverly

phrased question can extract it regardless of the instruction. Fix: filter at the ABAC/database query layer, before retrieval.

3. "Allow All" scope granted at setup, to be narrowed "later" → ❌ Not Secure

This is literally the Vercel/Context.ai root cause. "Later" rarely happens — over-broad scopes tend to persist indefinitely, turning any future compromise anywhere downstream into a worst-case scenario. Fix: least privilege from day one, not as a planned follow-up.

4. Audit log missing `on_behalf_of` → ❌ Not Secure

Without this field, you cannot distinguish a legitimate AI action from a manipulated one, and you can't attribute the action to the responsible human — the exact gap in the Obsidian Salesforce "wrong role invoked" case we discussed.

5. Banking chatbot drafts transfer, requires human confirmation before execution → ✅ Secure

This is the human-in-the-loop pattern applied correctly — matches the "draft vs. auto-send" and MFA-as-a-philosophy discussion. Even a fully manipulated AI can only produce a draft; a human catches anything wrong before it becomes irreversible.

6. Push-notification MFA with no send-rate limit → ❌ Not Secure

This is the exact condition that enabled the Uber breach and the Lapsus\$ group's repeated MFA-fatigue attacks. Fix: rate limit approval requests, and use number-matching instead of a blind approve/deny tap.

Self-Check: Can you explain these without notes?

If you can answer these out loud in your own words, Week 2 is genuinely solid:

- Why is `intersection` safer than `union` when an AI agent acts on a user's behalf?
- Why doesn't encryption solve the RAG data-leakage problem?
- What's the difference between object-level and property-level authorization?
- Why did the McDonald's breach have nothing to do with "AI going rogue"?
- What two fields make an audit log AI-aware, not just security-aware?

If any of these feel shaky, that's a good signal of exactly where to review before starting Phase 3 next session.